Increasing Reliability in Data Center Network Configuration

Arthur Davis, Tom Distler
NetApp
Boulder, Colorado, USA
Arthur.Davis@netapp.com, Tom.Distler@netapp.com

Abstract

In this paper, we describe the history and requirements of network configuration solutions for a large storage application based on a high availability cluster architecture with requirements for dynamic and reliable software-driven network configuration. Such requirements include:

- programmatic access to configure the majority of the Linux network stack,
- transaction-like semantics where multiple related configurations either fail or succeed as a unit,
- logic to monitor the system and, when it differs from the expected configuration, provide notifications and possibly attempt automatic repairs,
- and all of this without becoming entangled with application-specific expectations about behaviors and policies.

We expand upon the above set of core requirements and conclude with a description of a configuration system we are building at NetApp to meet these requirements.

Keywords

Linux, network, configuration, data-center, scale-out storage, reliability

Introduction

The network is at the core of any distributed system. Configuring and managing the network stack can be a complex task for these types of applications. This is especially true when implementing advanced networking features that require dynamic system modifications. This paper describes a design philosophy and network configuration system that addresses these issues.

Our ultimate goal is to create a distro-agnostic Linux networking configuration solution that addresses what we perceive as an unmet need in the community. We have surveyed many of the open source network configuration systems available for Linux and have been unable to find any solution that adequately meets our requirements.

The thoughts and designs presented in this paper are the culmination of years of work implementing highly-available distributed systems. Our current focus is the SolidFire scale-out storage solution [1] which is on its third generation of network configuration solutions. Our goal is to build a system that proves more robust, consistent, and extensible than what is currently available.

Scope

A full solution for network configuration in a product integrates many cooperating functions. These provide validations around how the product is intended to be used on the network and implement behaviors in the event of various network conditions in addition to allowing product users and automated processes to set up the kernel network stack to provide expected functionality.

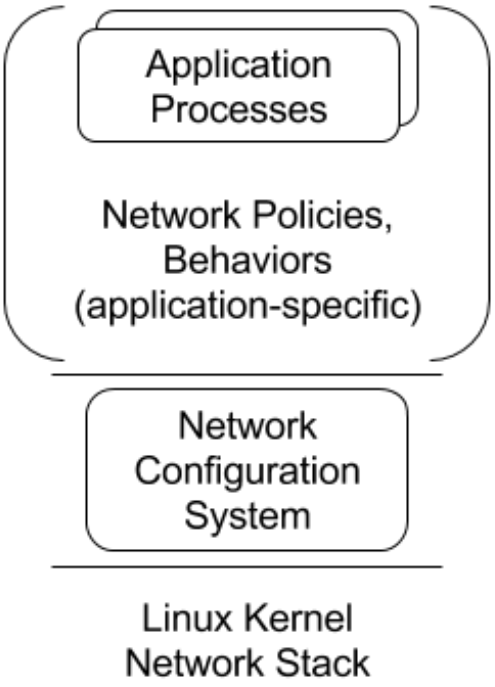


Figure 1: A simple decomposition of the functionality required to implement a robust networked application

In figure 1, we distinguish between the “Application” and the “Network Configuration System”. The application refers to the parts of the system that bear the responsibility for ensuring product requirements and functionality are implemented. The primary purpose of the network configuration system is to expose the kernel’s interfaces for configuring the

network stack and facilitate the application to efficiently implement product-specific network management functionality.

This paper discusses the lower-level portion of the network management solution referred to in the diagram as the network configuration system. It increases product reliability by supporting design patterns that allow application designers to focus on application functionality and behaviors in response to network events and conditions.

Focus on Application Design Patterns

We do not intend to define formal design patterns in the software engineering sense of the term. Rather, we refer to design patterns as expectations around how we desire to decompose the problems we are solving by implementing the application. This section informally describes those expectations by comparing the responsibilities of the application and the network configuration system (referred to as the NCS).

Configuration Ownership

The application owns the content of the network configuration. The NCS owns the responsibility to make sure the running system matches that expected configuration at all times. The application is able to update the configuration at any time. The NCS works to put the system into the expected state.

Multiple Configuration Actors

Many products have multiple processes that update the system's network configuration. The NCS supports multiple controllers by providing guarantees that each change set is handled independently and that separate read-modify-write operations do not overlap. This ensures that the system's network configuration moves from one consistent configuration state to another. The application must handle potential disagreements or contention between controllers over specific configuration changes.

Error Handling

When the application attempts to make a set of changes to the configuration, the NCS may be unable to apply all the changes because of errors encountered. As a result, it will revert the system to the previous (unchanged) state and report an error to the application. It is up to the application to implement proper functionality to handle this case.

Configuration Validation

The NCS does not impose any constraints or expectations beyond those already imposed by the Linux kernel configuration interfaces. It does not make determinations about correctness of any given configuration or use of Linux network capabilities but exposes the full configurability of the Linux network stack.

The application implements requirements about what kind of configuration is allowed. For example, does the product support VLANs? If so, how many? How does the product implement VRFs? What use case is it using VRFs to address? The NCS does not constrain these any more than the Linux kernel would do. The application is responsible for ensuring configuration correctness based on product requirements.

Event Handling

Some applications may want to implement behaviors that depend on certain network states or conditions (e.g. verifying that Internet access is available before proceeding with an operation or raising a fault to a larger network management system). The NCS supports this by monitoring for a discrepancy between the expected configuration (provided by the application) and the actual state of the system. It provides a notification when a discrepancy is detected. The application must integrate these notifications with additional logic and tests to implement appropriate behaviors.

Scoped Management

A product needs to be free to integrate functionality provided by many different sources. These will undoubtedly have different approaches to network configuration and will not all be able to use interfaces that are coordinated by the NCS. The NCS only manages configuration that is explicitly given to it by the application. It can see and return information about other network configuration, but it will not modify anything outside the scope of what the application configures it to manage. The application must ensure that network configuration ownership is partitioned to avoid conflicts.

Supportability

Products must support troubleshooting activities to diagnose problems on a live system and after problems have occurred. The NCS logs all changes made to network configuration. It also supports runtime interrogation of its internal state.

Architectural Details

This section describes the high-level architecture of a network configuration system designed to meet the requirements described in the preceding section.

Data Model

The data model refers to an organization of configuration data that the system maintains. Since we attempt to directly expose the Linux kernel network configuration interfaces, this organization also matches the organization of configuration data exposed by the kernel.

This same organization of data is exposed to users (or clients) of the network configuration system. In order to make a configuration change, a client will read some portion of the database, make some updates and provide a set of changes to update the configuration data managed by the NCS.

The NCS internally implements smart objects that correspond to each type of object in the data model (e.g. a specific type of network interface or a route table). Instances of these smart objects maintain the expected configuration for the kernel configuration entity that they represent and contain all the logic needed to enable the NCS to interact with the system to manage each kernel entity. These objects are instantiated as needed based on the desired configuration and are stored

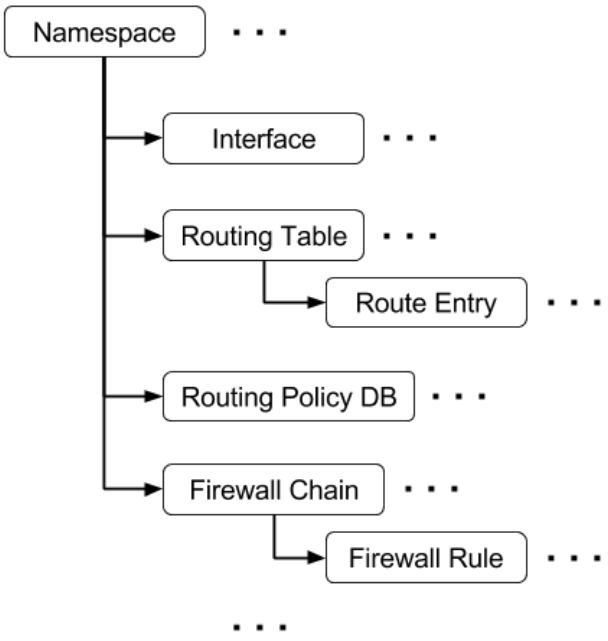


Figure 2: Depiction of the Network Configuration Data Model

in a versioned collection with the same organization as described by the data model. The version is incremented after any successful configuration update.

Source Objects

The concept of source objects decouples data representations and client communication protocols from the transaction engine described in the next section. This allows new sources of configuration changes to be added easily.

File sources are read on startup. Files from non-volatile storage are read to load persistent configuration. Files from volatile storage are used to recover from a crash or service restart and restore the internal state of the daemon.

A Unix domain socket source allows inter-process communication between the application and the NCS. This allows the use of built-in authorization via user ID, group ID, and network namespace.

Client Updates

This section provides a walkthrough of how several architectural elements are used to handle client requests to update the network configuration. The architectural elements are depicted in figure 3.

In this example, the request originates from a client via the unix domain socket source. The socket source acts on a clone of the configuration database and applies configuration changes from the request to the in-memory configuration database clone. In the diagram, the transaction engine refers to logic that drives the smart objects through a series of phases.

The validate phase applies syntactic and semantic validations. Syntactic validation checks that the individual values are valid for the configuration setting being changed (e.g. numerical ranges, enumeration values, etc.). Semantic validation checks dependencies between related configuration settings (e.g. making sure the nexthop router of a route is reachable via some interface). These validations catch errors that would prevent the NCS from successfully setting the configuration in the kernel.

Each smart object within the configuration database is responsible for validating its own part of the configuration. For example, a bond object is responsible for ensuring all bond-specific parameters are valid and that the specified slave devices are configured in a manner that is required by the bond. Successful validation indicates a high confidence by the NCS that the changes can be applied to the running system. If the validation step fails, the cloned configuration database is discarded and a messge is returned to the client.

The persist stage saves representations of the new configuration in several ways:

- writes any changes to the transaction log for traceability and debug;
- writes configuration data marked by the application as persistent to non-volatile storage (read at every startup to restore the in-memory configuration database);
- writes internal data to a crash recovery file on volatile storage (read at every startup when it's available).

After the data is persisted, the active configuration database is replaced with the cloned configuration database and the configuration version number is incremented. Actors both internal and external to the NCS see this as an atomic change. After this point, the NCS will no longer be able to roll back to the previous version.

Notice that no changes have yet been made to the linux kernel network stack. The NCS makes ACID transactional guarantees around updating the configuration database. These guarantees do not apply to the manner in which the running system is updated. The final phase of the transaction is the first time the NCS attempts to change the system and bring it to the requested state. Any changes that cannot be immediately applied will result in fault notifications being raised to clients that have registered for such notifications. Additional attempts will be made as detailed in the following section.

Monitoring and Repair

The NCS has a continuous background activity to monitor the state of the system and compare it with the expected state as defined by the active configuration database. Any disagreement between the system state and the expected configuration will be logged and will generate a fault notification to registered clients.

In addition to generating notifications, the NCS will also attempt to repair the problem. These discrepancies can occur for a variety of reasons. As discussed above, the NCS may not be able to apply a set of changes all at once. Another possibility is that a configuration was changed in error by faulty software or a user logged into the system. Whatever

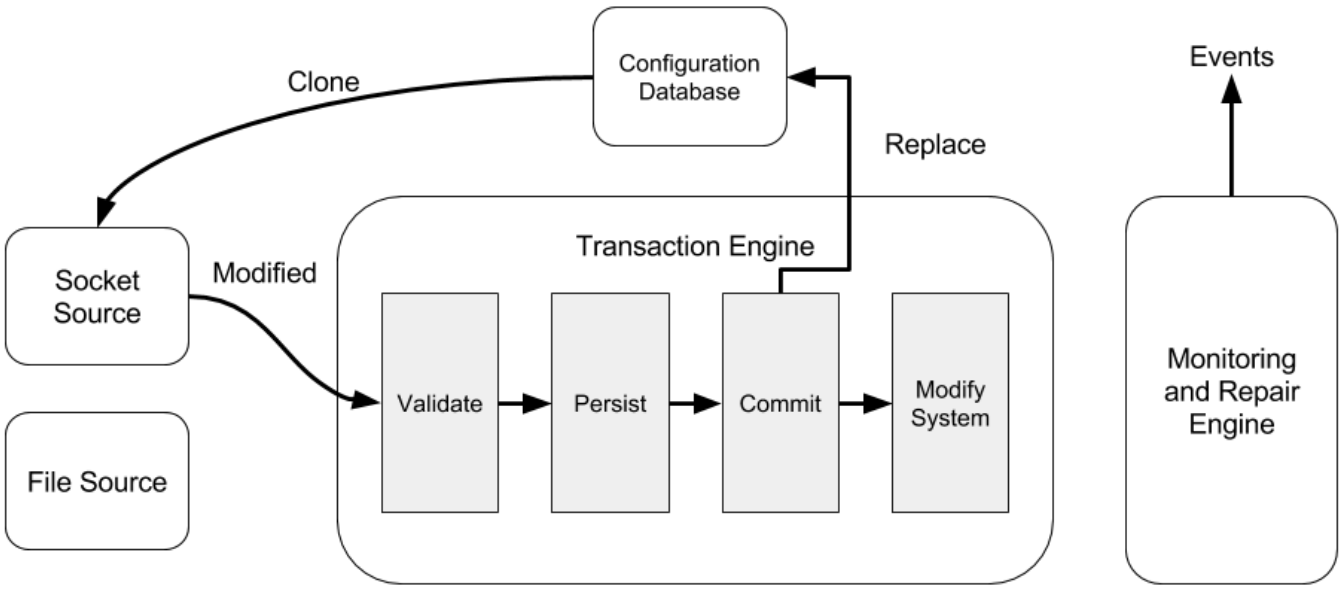


Figure 3: Network Configuration System Architecture

the cause, the NCS will attempt to reconfigure the system so that it matches the expected configuration. If it is successful, it will clear the fault notification.

Conclusion

In this paper, we have described a network configuration system that we believe addresses key issues that arise when building highly available, distributed systems. We have outlined key design principles that guide our approach to building distributed applications to improve reliability around network configurability. Finally, we described the high-level architecture of a network configuration system we are building to enable NetApp applications to take advantage of these principles.

The system we have described is one piece of a larger network ecosystem. The focus of this paper is only on local system network configuration. Building a full product will require additional network functionality and tools (e.g. that provide node discovery and automated configuration).

We welcome questions and discussion about the functionality discussed in this paper as well as how this can fit as part of a larger network ecosystem. Please contact us at the email addresses provided above.

Author Biographies

Arthur Davis is a senior-level software engineer working in the SolidFire business unit of NetApp. He has been implementing networking software for more than 15 years. This is the third time he has worked on a team implementing a network configuration system from scratch and is excited about the opportunity to share this one with a larger community as an open source project. His previous employers include Ericsson and Gogo.

Tom Distler is a senior-level software engineer working in the SolidFire business unit of NetApp. He is currently focused on improving network reliability and bringing advanced datacenter networking features to SolidFire. Much of his career has been focused on creating platform building blocks for high-availability distributed systems. His previous employers include Intel, Polycom, and Schneider Electric.

References

[] Dave Wright. Tech Field Day Presentation, April 25, 2014. Comparing Modern All-Flash Architectures. https://www.youtube.com/watch?v=AeaGCeJfNBg.